

Phase 3: Retrieval-Augmented Generation (RAG) Pipeline

This chapter covers **RAG (Retrieval-Augmented Generation)**, where we teach the system to augment LLM answers with external knowledge. We will cover document **loaders**, **text splitters**, **embeddings & vector stores**, **retrievers**, and then a **full RAG pipeline**. Each section builds on the previous, leading to an end-to-end RAG Q&A application.

Document Loaders

Concept and Theory: In LangChain, a **Document** is a standard format for any piece of text we work with. Each `Document` has a `pageContent` (the actual text) and a `metadata` object (additional info like source, page number, etc.) [11†L159-L167] . Document loaders are tools that read data from various sources (PDFs, text files, web pages, etc.) and return a list of `Document` objects. This unifies different data sources into the same format for later processing. For example, `TextLoader` reads plain text files, and `PDFLoader` reads PDF files, parsing each page into one or more `Document` s. Similarly, `CheerioWebBaseLoader` fetches a web page and turns its text into a `Document` . All loaders implement a standard interface with methods like `load()` (to load all documents) and `lazyLoad()` (to load documents one by one). You use `load()` when your data fits in memory; use `lazyLoad()` (a generator) for very large data sets to avoid memory spikes [16†L28-L36] [16†L35-L41] .

Loaders simplify the **ingestion** of external data. For example, `PDFLoader` will parse each PDF page into text automatically, saving you manual parsing effort [6†L44-L52] . Once loaded, you get an array of `Document` objects. For instance, after loading a text file you might see:

```
Document {
  pageContent: "This is the file content...",
  metadata: { source: "path/to/file.txt" }
}
```

As this shows, the `source` metadata stores the file path [22†L143-L151] .

Practical Lab: Let's try loading different documents step by step. First, install LangChain and any necessary dependencies. For text files we need only LangChain. For PDFs we need a PDF parsing library (e.g. `pdf-parse`). For web pages, we also need `cheerio` .

```
npm install langchain @langchain/openai @langchain/core
npm install @langchain/community pdf-parse # for PDFLoader
npm install cheerio # for web scraping
```

1. **Load a text file.** Create a text file `example.txt` with some content. In code, import and use `TextLoader` :

```
import { TextLoader } from "@langchain/classic/document_loaders/fs/text";
// Create a loader for our file
const loader = new TextLoader("example.txt");
// Load documents
const docs = await loader.load();
console.log(`Loaded ${docs.length} document(s).`);
console.log("First document content:", docs[0].pageContent.trim());
console.log("Metadata:", docs[0].metadata);
```

Explanation: We import `TextLoader` from `LangChain`. We call `new TextLoader("example.txt")` to point it at our file. The `load()` method reads the file and returns a list of `Document` objects. We print out the number of documents (for a single small text file it will be 1) and inspect the first document's content and metadata. The output should show the text of the file and something like `{ source: "example.txt" }`. A common mistake is forgetting `await` on `load()`, which returns a promise.

2. **Load a PDF file.** Place a PDF (e.g. `sample.pdf`) in your project. Use `PDFLoader` :

```
import { PDFLoader } from "@langchain/community/document_loaders/fs/pdf";
// Create loader for PDF. By default, each page is a separate Document.
const pdfLoader = new PDFLoader("sample.pdf");
const pdfDocs = await pdfLoader.load();
console.log(`Loaded ${pdfDocs.length} page(s) from PDF.`);
// Show the first 100 characters of page 1
console.log(pdfDocs[0].pageContent.slice(0, 100));
console.log("Metadata of first page:", pdfDocs[0].metadata);
```

Explanation: `PDFLoader` comes from the `@langchain/community` package. By default it splits each page into a separate `Document` (you can disable that with `splitPages: false`). We print how many documents were returned (one per page) and peek at the first page's text. The metadata will include the source file path and the page number. A common issue is forgetting to install `pdf-parse`; if the loader fails to extract text, ensure `pdf-parse` is installed.

3. **Load a web page.** Use `CheerioWebBaseLoader` :

```
import { CheerioWebBaseLoader } from "langchain/document_loaders/web/cheerio";
const url = "https://en.wikipedia.org/wiki/LangChain";
const webLoader = new CheerioWebBaseLoader(url);
const webDocs = await webLoader.load();
console.log(`Loaded ${webDocs.length} document(s) from web page.`);
console.log("Snippet:", webDocs[0].pageContent.slice(0, 100));
console.log("Source metadata:", webDocs[0].metadata);
```

Explanation: We point the loader at a URL. When we call `load()`, Cheerio fetches the page and extracts the visible text. The result is usually one `Document` containing the main text. We print its first 100 characters and metadata. Make sure to `npm install cheerio` as shown above; otherwise the loader cannot run.

4. **Load multiple files with `DirectoryLoader`**. Suppose you have a folder `docs/` with several `.txt`, `.pdf`, etc. files. You can use `DirectoryLoader` to batch-load them:

```
import { DirectoryLoader } from "@langchain/classic/document_loaders/fs/directory";
import { TextLoader } from "@langchain/classic/document_loaders/fs/text";
import { PDFLoader } from "@langchain/community/document_loaders/fs/pdf";
// Define folder and extension mappings
const dirLoader = new DirectoryLoader("docs", {
  ".txt": (path) => new TextLoader(path),
  ".pdf": (path) => new PDFLoader(path),
});
const dirDocs = await dirLoader.load();
console.log(`Loaded ${dirDocs.length} documents from directory.`);
console.log("Example metadata:", dirDocs[0].metadata);
```

Explanation: `DirectoryLoader` takes a directory path and a mapping from file extensions to loader factories. It then runs the appropriate loader for each file and returns all documents in one array. In this example, we load any `.txt` with `TextLoader` and `.pdf` with `PDFLoader`. We then print how many documents resulted. A tip: if your directory is huge, consider using `dirLoader.lazyLoad()` in a loop to process one doc at a time to save memory.

Key Takeaways: Document loaders convert raw files or web content into LangChain `Document` objects (with `.pageContent` and `.metadata`) [\[11†L159-L167\]](#). Use `load()` to get all documents at once (good for small data), or `lazyLoad()` for memory efficiency [\[16†L28-L36\]](#) [\[16†L35-L41\]](#). LangChain has many built-in loaders (PDF, text, CSV, JSON, HTML, etc.) [\[6†L44-L52\]](#) [\[6†L51-L57\]](#). Always inspect the `Document` metadata (e.g. source, page) to verify correct loading.

Practice Questions: What fields does a LangChain `Document` have? (Answer: `pageContent`, `metadata`, and optionally `id` [\[11†L159-L167\]](#).) When would you use `lazyLoad()` instead of `load()`? (Answer: when data is very large, to avoid memory overload [\[16†L35-L41\]](#).)

Exercises: Try loading a folder with mixed file types using `DirectoryLoader` and printing out each document's source and first sentence. Intentionally omit installing `pdf-parse` and observe the error when loading a PDF, then fix it by installing the missing library.

Text Splitters

Concept and Theory: Large documents often exceed a model's context window. **Text splitters** break a long text into smaller chunks. Two key parameters are `chunk_size` (maximum length) and

chunk_overlap (how much each chunk overlaps with the next, to preserve context). For example, a chunk size of 500 tokens with overlap 50 means each chunk shares 50 tokens with its neighbor. Overlap helps ensure important information isn't cut off at boundaries.

LangChain provides splitters like **CharacterTextSplitter** and **RecursiveCharacterTextSplitter**. *Recursive* splitter is smart: it tries to split at logical boundaries (like paragraphs or sentences) rather than cutting mid-sentence. It attempts larger units first (paragraphs), then if still too big, splits into smaller units (sentences or words) [35†L109-L114] . This balances preserving context with fitting the chunk size. If no higher-level break is found, it splits at the character or word level. Typically, start with a `RecursiveCharacterTextSplitter(chunkSize, chunkOverlap)` to get good splits [35†L89-L97] [35†L109-L114] .

There are also **length-based splitters**: e.g. token-based (split by token count) or character-based. Length-based splitting is straightforward and ensures uniform chunk sizes [35†L129-L137] . For instance, you might use `CharacterTextSplitter.fromTiktokenEncoder({ encodingName: "cl100k_base", chunkSize: 500 })` to split by tokens. Other advanced splitters exist for semantic boundaries or structured data (e.g. splitting code by functions), but as beginners we'll focus on character/sentence splitters.

Practical Lab: Let's split some text and see the chunks. We'll use a small example string for clarity.

1. Simple split with overlap:

```
import { CharacterTextSplitter } from "@langchain/classic/
text_splitter";
import { Document } from "@langchain/core/documents";
// Example text (simulate a long paragraph)
const longText =
  "LangChain is a framework for AI. It helps combine LLMs with data. " +
  "Text splitters break this into chunks. Each chunk can overlap next
  one.";
const docs = [ new Document({ pageContent: longText, metadata: {} }) ];
// Split with chunk size 50 chars and overlap 10 chars
const splitter = new CharacterTextSplitter({ chunkSize: 50,
chunkOverlap: 10 });
const splitDocs = await splitter.splitDocuments(docs);
console.log(`Number of chunks: ${splitDocs.length}`);
splitDocs.forEach((d, i) => {
  console.log(`Chunk ${i+1} (${d.pageContent.length} chars):`,
  d.pageContent);
});
```

Explanation: We created one `Document` from a sample string. The splitter is set to 50-character chunks with 10-character overlap. After splitting, we log each chunk. You will see that the last 10 characters of chunk 1 are repeated at the start of chunk 2. This overlap helps the model "remember" end-of-chunk context. Common pitfalls: if you use too large overlap, chunks will have a lot of duplicated text, wasting tokens. If too small, context might be lost at boundaries.

2. Recursive splitting on structured text:

```
import { RecursiveCharacterTextSplitter } from "@langchain/classic/
text_splitter";
// Simulate a document with paragraphs
const structuredText = "Paragraph one is here. It has some sentences. "
+
  "Paragraph two comes here. It also has content.";
const structuredDoc = [ new Document({ pageContent: structuredText,
metadata: {} }) ];
// Split into ~30-char chunks, overlap 5
const recursiveSplitter = new RecursiveCharacterTextSplitter({
chunkSize: 30, chunkOverlap: 5 });
const structuredChunks = await
recursiveSplitter.splitDocuments(structuredDoc);
structuredChunks.forEach((d,i) => {
  console.log(`Recursive Chunk ${i+1}:`, d.pageContent);
});
```

Explanation: We use `RecursiveCharacterTextSplitter` with a small chunk size. It will try to keep sentences intact. You should see output where sentences are not split in the middle. For instance, "Paragraph one is here." might be one chunk. A newbie error is forgetting that `splitDocuments` is async (it returns a promise), so you need `await`.

3. **Experiment: overlap effects.** Change `chunkOverlap` to 0 and rerun the first example. Notice how chunks become disjoint, possibly losing important context. Then try increasing overlap to 20 and see duplicated phrases. This illustrates the trade-off [35†L129-L137] .

Key Takeaways: Splitting documents ensures each chunk is within token limits. Use `RecursiveCharacterTextSplitter` to preserve sentence/paragraph boundaries [35†L109-L114] . The `chunkSize` sets how big each piece is; `chunkOverlap` sets how much they share (overlap) to maintain continuity. Length-based splitting (character or token) is straightforward [35†L129-L137] . Always choose chunk sizes considering your LLM's context window.

Practice Questions: Why do we split documents before embedding? (Answer: to fit model limits and allow retrieval on smaller pieces.) What does `chunkOverlap` do? (Answer: repeats some text at boundaries to keep context.)

Exercises: Take a medium-length article (copy into a string) and experiment with different splitters: compare pure `CharacterTextSplitter` vs `RecursiveCharacterTextSplitter` . Print out how many chunks you get and inspect where splits occur. Try splitting some code or JSON text to see where naive splitting might break structure.

Embeddings & Vector Stores

Concept and Theory: Embeddings convert text into high-dimensional numerical **vectors** that capture semantic meaning. In LangChain, an embedding model (e.g. OpenAI's) generates a vector for each text. These vectors allow us to measure similarity: for a new query, we compute its vector and find which document vectors are closest. Common similarity metrics are **cosine similarity** (angle between vectors) or **dot product**. Cosine ignores vector length, focusing on direction; dot product also considers magnitude.

In practice, we generate embeddings for each document chunk (from our splitters) and store them in a **vector store** (a database for vectors). LangChain offers many stores:

- **MemoryVectorStore** (in-memory, ephemeral) [41†L94-L100] , good for demos;
- **FAISS** (a fast in-memory C++ library, often with GPUs);
- **Chroma** (local, persistent store);
- **Pinecone** (cloud-based vector DB);
- others (Weaviate, Qdrant, etc.).

Example: using the OpenAIEmbeddings model, we can get vectors for text. In LangChain we do `new OpenAIEmbeddings({model: "text-embedding-3-large"})` to select a model [11†L218-L226] . After embedding documents, the vector store is queried by adding a query vector and performing a nearest-neighbor search.

Embedding documents and storing them is known as **indexing**. During **query time**, we embed the user's query and do a **similarity search** in the vector store to find the most relevant chunks [11†L192-L200] . This returns the top-k nearest documents.

Practical Lab: We will embed some sample text and use an in-memory vector store to retrieve similar content.

1. Embed sentences and compare similarity:

```
import { OpenAIEmbeddings } from "@langchain/openai";
import { MemoryVectorStore } from "@langchain/classic/vectorstores/memory";
import { Document } from "@langchain/core/documents";
// Prepare documents
const docs = [
  new Document({ pageContent: "Apples are red fruits", metadata: {} }),
  new Document({ pageContent: "Bananas are yellow fruits", metadata: {} }),
  new Document({ pageContent: "Roses are beautiful flowers", metadata: {} })
];
// Initialize embedding model
const embeddings = new OpenAIEmbeddings({ model: "text-embedding-3-large" });
// Create in-memory vector store and add documents (this will embed them)
const store = new MemoryVectorStore(embeddings);
await store.addDocuments(docs);
console.log("Documents indexed.");
// Check vector dimensions for one document (debug)
const vector = await embeddings.embedQuery("sample");
console.log(`Embedding vector length: ${vector.length}`);
```

Explanation: We create three simple documents. We instantiate `OpenAIEmbeddings` (you'll need a valid API key in `OPENAI_API_KEY`). We then make a `MemoryVectorStore`, passing the embedding model to it. Calling `addDocuments(docs)` computes embeddings for each

document and stores them internally. We print when indexing is done. The debug log shows the embedding vector's dimensionality (for `text-embedding-3-large`, it should be 1536 dimensions). If you get an authentication error, check your API key.

2. Similarity search query:

```
const query = "Which fruit is red?";
const k = 2;
// Simple search:
const results = await store.similaritySearch(query, k);
console.log(`Top ${k} results for "${query}":`);
results.forEach((doc, i) => {
  console.log(`${i+1}. ${doc.pageContent}`);
});
```

Explanation: We ask the store "Which fruit is red?" The code calls `similaritySearch`, which embeds the query and finds the top 2 closest documents. The output should list "Apples are red fruits" first, since it's highly relevant, and likely "Bananas are yellow fruits" or the flower line second. We print the content of each returned Document. A common mistake is not awaiting the async calls.

3. **Persistent vs In-Memory:** The `MemoryVectorStore` does not save to disk. To experiment with persistence, try using `Chroma` (if installed) or note that `MemoryVectorStore`'s scope is just during runtime.

Key Takeaways: We convert text to numeric vectors (embeddings) to do semantic search. With LangChain's `OpenAIEmbeddings`, you can embed documents `【11†L218-L226】`. A **vector store** holds these vectors and allows similarity search. For example, the in-memory vector store uses cosine similarity by default `【41†L94-L100】`. To query, we embed the query and call `similaritySearch("...", k)` on the store. This returns the top-k most similar documents. The code example above shows the output format of retrieved `Document`s.

Practice Questions: What does an embedding model do? (Answer: converts text to a numeric vector representing meaning.) Why use a vector store? (Answer: to efficiently find texts semantically similar to a query by comparing embedding vectors.) What similarity metric does `MemoryVectorStore` use by default? (Answer: cosine similarity `【41†L94-L100】`.)

Exercises: Experiment by adding more documents to the store (e.g. different fruits or objects) and issuing queries. Try `similaritySearchWithScore(query, k)` to get similarity scores. Observe how changing `k` (the number of results) or query phrasing affects results.

Retrievers

Concept and Theory: A **retriever** is a component that, given a query, fetches relevant `Document`s from a vector store. In LangChain, the simplest retriever is the `VectorStoreRetriever`, which essentially wraps a vector store search. It takes a `k` (how many docs to return) and returns the closest matches. Under the hood, it calls the store's similarity search. For example, `vectorStore.asRetriever(2)` creates a retriever that returns 2 results.

Beyond plain search, LangChain offers advanced retrieval strategies:

- **Maximal Marginal Relevance (MMR):** This method first retrieves more candidates (say top 10) and then re-ranks them to balance relevance **and diversity**. MMR helps avoid returning nearly duplicate passages. In LangChain's in-memory store, you can enable MMR by specifying `searchType: "mmr"` and `searchKwargs.fetchK`, e.g. fetch 10, then take top 2 diverse results [\[58†L264-L272\]](#) .
- **Multi-Query Retriever:** This strategy uses an LLM to rewrite the query into multiple related queries, retrieves results for each, and combines them. It helps when the original query is ambiguous or multi-faceted. (Implementing this would involve using a chat model to generate sub-queries, then merging results. We won't code it here, but it's available in LangChain.)
- **Contextual Compression Retriever:** Here, after retrieving documents, an LLM is used to filter or extract only the relevant parts of each document with respect to the query [\[53†L115-L124\]](#) . This "compresses" the documents to remove irrelevant bits. It's useful to reduce token usage and focus the LLM on just the needed info.

In summary, a retriever produces `Document` s most relevant to the query [\[63†L98-L102\]](#) . You can use simple top-k search or richer methods (MMR, multi-query, compression) depending on your needs.

Practical Lab: We continue with the vector store from before and perform retrieval using different methods.

1. Basic top-k retrieval:

```
const retriever = store.asRetriever(2); // create retriever for top 2
const retrieved = await retriever.invoke("red fruit");
console.log("Retriever returned documents:");
retrieved.forEach((doc, i) => {
  console.log(`${i+1}. ${doc.pageContent} [${
    JSON.stringify(doc.metadata)}]`);
});
```

Explanation: We call `asRetriever(2)` , which wraps the vector store. Then `invoke("red fruit")` runs the query through the retriever. The output is an array of `Document` s. We print their contents. You should see the most relevant document(s) we indexed earlier. If none match, check that your `addDocuments` step succeeded.

2. MMR Retrieval:

```
// Create an MMR retriever that fetches top 10 then returns 2
const mmrRetriever = store.asRetriever({
  searchType: "mmr",
  searchKwargs: { fetchK: 10 },
  k: 2
});
const mmrResults = await mmrRetriever.invoke("fruit");
```



```
console.log("MMR retriever results:");
mmrResults.forEach((doc, i) => {
  console.log(`${i+1}. ${doc.pageContent}`);
});
```

Explanation: Here we enabled MMR. We ask for top 2 (`k:2`) but allow it to fetch up to 10 candidates first (`fetchK:10`) 【58†L264-L272】 . The MMR algorithm will pick the 2 that are both relevant and diverse. Try comparing with the plain retriever results. If your data is small, you may not notice a difference yet, but the code structure shows how to set up MMR.

3. **(Optional) Multi-Query and Compression:** These require LLM chains. We won't code them now, but note that LangChain provides `MultiQueryRetriever` and `ContextualCompressionRetriever` classes. For example, a `ContextualCompressionRetriever` can be created by wrapping a base retriever with a `DocumentCompressor` (such as an LLM chain extractor) 【53†L115-L124】 .

Key Takeaways: A retriever selects relevant documents for a query, often via similarity search in the vector store 【63†L98-L102】 . The simplest is top-k similarity. LangChain also supports MMR for diverse results 【58†L264-L272】 . Advanced retrievers (multi-query, compression) use LLMs for query rewriting or doc filtering. In code, using `vectorStore.asRetriever(...)` and then `invoke(query)` is a quick way to retrieve.

Practice Questions: What is the purpose of MMR in retrieval? (Answer: to provide diverse results by re-ranking an initial larger result set 【58†L264-L272】 .) How do you create a retriever that returns 3 documents? (Answer: `vectorStore.asRetriever(3)` .)

Exercises: Using your vector store, try retrieving with different `k` and notice how that changes the number of results. If you have multiple documents, experiment with MMR as shown above and see if it changes which documents are returned.

RAG – Full Pipeline

Concept and Theory: A **RAG (Retrieval-Augmented Generation)** system combines retrieval with generation. The typical **pipeline** has two phases 【63†L82-L90】 【63†L98-L102】 :

- **Indexing (offline):** Load your documents (with loaders) → split them into chunks → embed each chunk → store the vectors in a vector database. Essentially, prepare the knowledge base ahead of time.
- **Retrieval and Answering (runtime):** When a user asks a query, convert it to a vector, retrieve the top-k relevant document chunks, and then feed those chunks (plus the query) into the LLM to **generate** an answer. This way, the answer is grounded in actual source text, reducing hallucination 【63†L98-L102】 .

There are different strategies for assembling the retrieved context into the LLM prompt 【63†L69-L77】 :

- **Stuff (concatenate):** Simply concatenate all retrieved text and append the user's question. This is simplest but can hit context limits if there's a lot of text.

- **Map-Reduce:** First, generate an intermediate answer for each chunk separately (the “map” step), then combine all intermediate answers into a final answer (the “reduce” step). Useful when many chunks are needed.
- **Refine:** Start by answering with one chunk, then iteratively “refine” that answer using each additional chunk. This can produce more nuanced answers if information is spread across chunks.

A conversational RAG system might also incorporate **chat memory**, keeping track of past user turns, but that extends beyond static RAG.

We can also return **source citations** by attaching each chunk’s metadata. For example, including the `source` field from `Document.metadata` with the answer to show where facts came from.

Practical Lab: Let's build a simple RAG-based Q&A using the pieces above. We'll use the in-memory store and a Chat model for generation.

```
import { ChatOpenAI } from "@langchain/openai";
import { createRetrievalChain } from "langchain/chains/retrieval";

// (Assume `store` is our MemoryVectorStore already indexed with documents
// from before.)

// 1. Set up the chat model (GPT-based, temperature=0 for deterministic
// answers)
const llm = new ChatOpenAI({ modelName: "gpt-3.5-turbo", temperature: 0 });

// 2. Create a retrieval QA chain (this handles combining retrieval +
// generation)
const qaChain = createRetrievalChain({
  llm: llm,
  retriever: store.asRetriever(2) // use top-2 docs for context
});

// 3. Ask a question
const question = "What is red and commonly eaten in pies?";
const result = await qaChain.invoke({ input: question });
console.log("Answer:", result);
```

Explanation: We imported `ChatOpenAI` as our LLM and created `qaChain` via `createRetrievalChain` (which under the hood retrieves context and then calls the model). We set the retriever to return 2 docs. Then we `invoke` the chain with our question. The output (`result`) is the model’s answer string. In a real application, you’d examine or return this.

Under the hood, LangChain did this:

- Embed “What is red and commonly eaten in pies?”
- Retrieve the top-2 document chunks (from our vector store, likely “Apples are red...” etc.),
- Insert those chunks as context into the chat prompt,
- Call the OpenAI model to generate an answer.

For our example, it might answer: “Apples are red and commonly eaten in pies.”

To **return sources**, you could modify the chain or manually attach metadata. For example, after getting the docs from `qaChain`, you might log:

```
const docsUsed = await store.similaritySearch(question, 2);
docsUsed.forEach(doc => console.log("Source:", doc.metadata.source));
```

This shows where each piece came from.

Key Takeaways: A complete RAG system indexes documents ahead of time (load → split → embed → store), and at query time retrieves relevant chunks and passes them to the LLM [63†L98-L102]. Common prompt-building strategies include **Stuff** (concatenate) and **Map-Reduce/Refine** (process chunks individually then combine) [63†L69-L77]. Using LangChain's `createRetrievalChain` simplifies this: it handles retrieval and asks the model to answer. Remember to include source attributions from `Document.metadata` if you need traceability.

Practice Questions: What are the main steps of a RAG pipeline? (Answer: indexing [load, split, embed, store] and retrieval+generation [query embed, retrieve, prompt, answer].) What is the difference between “stuff” and “map-reduce” chains? (Answer: Stuff concatenates all retrieved text in one prompt; map-reduce answers each part then combines.)

Exercises: Using a small text or PDF of your choice, build a simple Q&A bot: index the document, then loop asking the user questions and printing answers. Try comparing a direct Stuff approach (send all context at once) versus using `createRetrievalChain` (which by default uses a retrieval strategy). If your text is large, ensure it's split so the model isn't overwhelmed.

Final Project: End-to-End RAG Application

For a final project, combine all the above into a complete RAG-based chatbot or Q&A system:

1. **Data Preparation:** Choose a set of documents (e.g. a PDF book, a folder of articles, or scraped web pages). Use **Document Loaders** to ingest them, and **Text Splitters** to chunk them into manageable pieces.
2. **Indexing:** Use an embedding model (e.g. OpenAIEmbeddings) and a vector store (e.g. Chroma or MemoryVectorStore) to index all chunks. Verify that each chunk's `metadata.source` indicates the origin (file path or URL).
3. **Query Interface:** Build a loop or function where a user can input a question. In that loop, take the question, use the vector store retriever to find the top-**k** relevant chunks, and feed those to an LLM (via a chain).
4. **Answer Generation:** Use a Chat model (e.g. GPT) in a retrieval chain (`createRetrievalChain`) or manual chain to produce an answer. Display the answer along with any source citations (e.g. print the URLs or filenames from the retrieved chunks' metadata).

5. **Variation:** Experiment with different chain strategies. For example, for long answers, try a map-reduce approach: for each retrieved chunk, generate a partial answer, then combine them in a final prompt.
6. **Evaluation:** Test your system with questions about the document content. Check if the answers are correct and grounded in the sources.

Mini-Project Tip: You could make a simple web interface (or a command-line prompt loop) for this QA system. For instance, upload a course PDF or a set of meeting notes and ask questions interactively.

By completing this project, you will have implemented all of Phase 3: loading data, splitting text, building embeddings and a vector store, retrieving relevant context, and generating answers. This showcases a full Retrieval-Augmented Generation application using NodeJS/TypeScript and LangChain.
